Stage 4 Demo Script

When: Wednesday, October 28, 2026
Length: 5 to 8 minutes (aim for 7)
Slides: `slides.pdf` in this folder. Notes for each slide are in `speaker-notes.md`; the full spoken script is in `presentation-script.md`.

The assignment requires four things in the demo: real data in the database, insert/update/delete, at least one search query, and the plan for the advanced functions. Each step below is labeled with the requirement it covers.

Before the Talk

The full word-for-word script, including when to launch the app, is in `presentation-script.md`. In short, from `~/Desktop/DB`:

- [ ] Night before: `./demo.sh setup <file>.sqlite` loads the real data and prints the demo farm, harvest date and market product to use
- [ ] After each rehearsal: `./demo.sh reset`
- [ ] 15 minutes before: `./demo.sh` checks the database, starts the app and opens the three tabs (home, demo farm, search). Leave the terminal open.
- [ ] Zoom the browser to 125% so the back of the room can read it
- [ ] Close other windows and notifications

Run of Show

Time	Slide or page	What to do
0:00	Slides 1-2	Introduce the team and the idea: three kinds of data joined on the taxon.
0:45	Slides 3-4	Real data and the schema. Keep it short; the app shows the rest.
1:45	Slide 5	"Let's see it running." Switch to the browser.
1:50	App	Steps 1-6 below.
5:30	Slide 6	The four queries; explain the multi-table join.
6:00	Slides 7-8	Advanced functions and timeline.
7:00	Slide 9	Questions.

Live Demo Steps

Step 1: Real data (requirement: real data in the database)

Home page `/`.

> "This is the live PostgreSQL database. 33,667 auction prices from two Ohio auctions, 2,993 taxonomy nodes, 107 real farms. The second table shows that 86% of prices already match a product. The rest is what Advanced Function 3 fixes."

Step 2: Insert (requirement: insert)

On the demo farm's overview:

1. **Crop catalog** tab, then search `onion`. Point out that results are ranked by how many auction lots each has. Click **Add** on *Allium cepa*.
2. **Overview**, then **Add a plot**: name `North field`, area `1200`. Click **Add plot**.
3. **Record an event**: Harvest, onion, plot North field, market product: Onions, date **2025-08-12** (or the month you checked), quantity `40`, unit `lb`. Click **Record event**.

> "Each of these is an INSERT with bound parameters, so user input never touches the SQL text."

Step 3: A database rule in action (requirement: insert, plus constraints)

Record an event again, enter quantity `10` and leave the unit blank, then submit.

> "That message comes from a CHECK constraint in the schema, not from application code. Any page that writes an event is held to it."

Step 4: Update (requirement: update)

Activity history, then **Edit** on the harvest. Change the quantity to `55` and save.

Step 5: Search and the multi-table join (requirement: search)

1. **Harvest vs. market** tab:

> "For each harvest, this joins the event log, farm, event kind, taxonomy, products, product aliases and auction lots, and returns what onion sold for within 30 days of our harvest date. It matches on the taxon, so every onion lot counts however the auction worded it."

1. Click **Crop search** in the header, search `onion`, open *Allium cepa*:

> "The seasonal price curve: floor, mean and ceiling per peck in every month across all eight seasons, plus how many seasons back each month. The links switch units."

Step 6: Delete (requirement: delete)

1. Back to the farm **Overview**. **Delete** the plot North field and confirm.
2. Open **Activity history**: the harvest is still there, with no plot.

> "Deleting a plot uses ON DELETE SET NULL, so the farm keeps its harvest history."

1. **Delete** the harvest event to finish. Optional: remove onion from the catalog.

If Something Goes Wrong

- **App won't start:** check that `DATABASE_URL` points at the loaded database and that PostgreSQL is running.
- **Harvest vs. market is empty:** the harvest date has no onion lots within 30 days. Edit the event's date to a month that had lots on the price curve.
- **Stuck for time:** skip Step 3 and the price curve. Insert, update, delete and the join are what's required.

Likely Questions

Question	Answer
Why is the roll-up "advanced"?	The user can start at any rank and the number of levels below it varies. A fixed number of joins can't express it; a recursive CTE walks the subtree in one statement.
Why not just match auction labels by substring?	"Wild garlic" is not garlic, and when two products match equally, picking whichever row comes first gives a wrong match. The ladder stops and reports ambiguity instead.
Where will grower data come from?	Real users. The site goes public the week of November 9, at least a week before the final demo, and we'll email you when it's live.
How do you prevent SQL injection?	Every value is a bound parameter. The SQL text never contains user input.
Why PostgreSQL rather than MySQL?	Recursive CTEs and window functions, which Advanced Functions 1 and 2 depend on, plus strict CHECK enforcement.